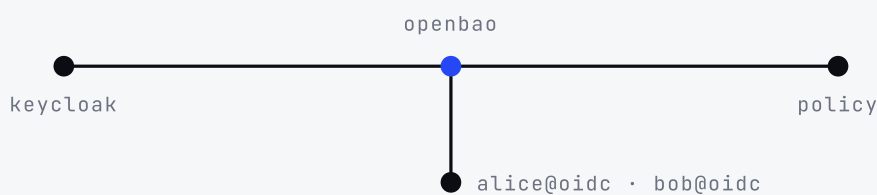


KEYCLOAK AND OPENBAO

Keycloak in the cluster, OIDC login for OpenBao, groups
→ policies – admin access without a password in
OpenBao.



Keycloak and OpenBao

Keycloak in the cluster via operator, a realm as code, OIDC login for OpenBao – and groups that become policies instead of passwords that live in OpenBao

Thomas Zachmann

17 September 2026

Contents

What this is about

- Why by hand, and why without AI

- License and liability

- A word about the values in this guide

- Who this is for

- The building blocks

- The architecture in one picture

Part I – Keycloak in the cluster

- Operator

- Database

- TLS from our own CA

- The instance

Part II – The realm as code

- The proof in the token

Part III – OIDC in OpenBao

- Auth method and configuration

- Two roles

- Policies

- Group → external group → policy

Part IV – The proof

- Login with the ID tokens

- What the rights do

- What OpenBao created

- The browser route

Part V – What went wrong, and why

Part VI – Operation

- Retiring the temporary admin

- Realm changes

- Removing the test role

- Nº 1, Part VI, re-read

- What is still open

Appendix A – All commands

Appendix B – Glossary

What this is about

Up to here the OpenBao in the cluster had two kinds of access: the root token and Kubernetes auth for workloads. For humans there was nothing – and N° 1 recommended userpass as an interim step, a password that lives in OpenBao itself. That is better than root, but it is a second password next to the one you already have, without MFA, without central lockout, without groups.

This guide builds the access you expect in a company: an identity provider – **Keycloak** – manages users and groups, OpenBao trusts it via OIDC, and the group membership in Keycloak decides which policies a token gets. Whoever is removed from the group `openbao-admins` is no admin on the next login – without anybody touching OpenBao.

Keycloak itself is deployed in the cluster via operator, with a PostgreSQL database from CloudNativePG and a TLS certificate from the CA of N° 5. The realm arrives as a `KeycloakRealmImport`, that is, as code. The login is proven without a browser – with an ID token from the password grant – and the browser route for UI and CLI is documented.

It is the eighth part of a series. Four errors came up; two of them lay in Keycloak 26.7, one in our own PKI role from N° 5.

Why by hand, and why without AI

OIDC has many places where two values have to match: issuer and discovery URL, `aud` and `bound_audiences`, redirect URI in the client and in the role, claim name in the mapper and in `groups_claim`. An AI sets them consistently – and when one does not match, it does not help finding it, because OpenBao's error message only says `error validating token`.

Tools like [nyrvex](#), which generates the configuration of an AI platform's secret store and identity provider, take these steps off your hands later. You should have walked them yourself once, to be able to judge what was generated.

The yardstick: whoever has worked through this guide can draw, on a blank sheet, the path a group membership takes from Keycloak to the `identity_policies` of an OpenBao token – through which four objects, and which of them lives in which system.

License and liability

This guide is licensed under CC BY 4.0: it may be copied, shared and adapted, commercially too, as long as the author is credited. It is provided as is, without warranty. Everything in it was carried out in a development environment; whoever reproduces it elsewhere does so at their own risk.

A word about the values in this guide

Client secret (openbao-client-secret-123) and user passwords (alice123, bob123) are development values and printed on purpose. The realm manifest has `directAccessGrantsEnabled: true` so that the test works without a browser – in a production configuration that is off.

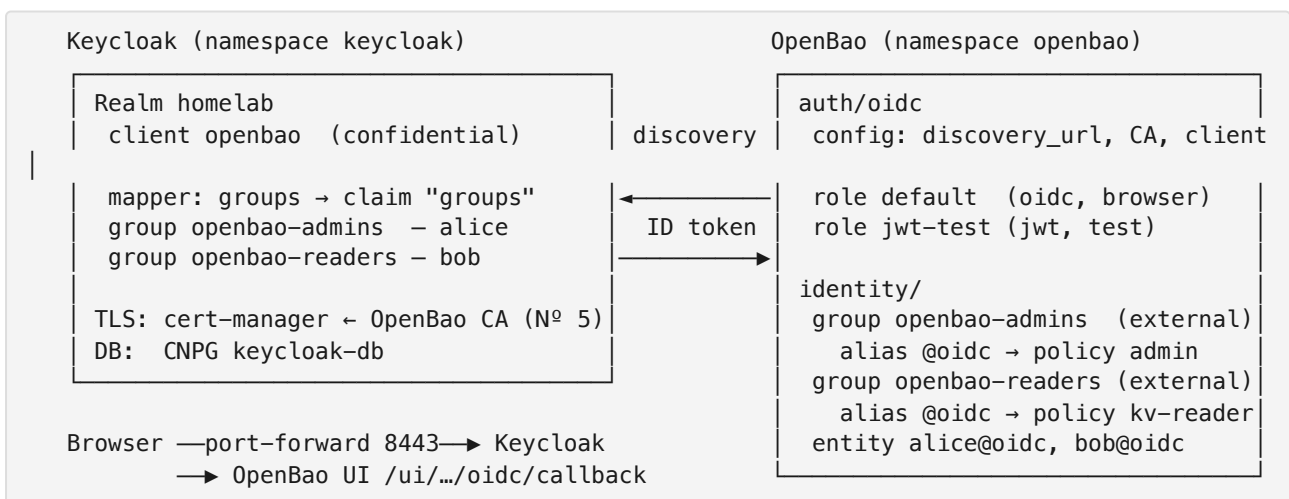
Who this is for

Readers who know N° 1 and understand OIDC in outline (issuer, client, ID token, claims). Keycloak experience helps but is not required – everything needed is there as a manifest. OpenBao basics are in my book **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice); for Keycloak itself there is *Keycloak in Practice* from the same series.

The building blocks

Component	Version	Role
Keycloak operator + Keycloak	26.7.4	the identity provider, realm as CR
CloudNativePG	operator 0.29	PostgreSQL for Keycloak, one Cluster object
cert-manager + OpenBao CA (N° 5)	v1.20.2	TLS certificate for Keycloak
OpenBao in the cluster (N° 1)	2.6.2	auth method oidc, identity groups

The architecture in one picture



Three things to take from the picture:

1. **Four objects carry the group.** The group in Keycloak, the mapper that writes it into the token, the external group in OpenBao, and the alias that connects the two – bound to the accessor of the auth method. If one is missing, the login succeeds, but without policies.

2. **The hostname is the Service name.** Keycloak terminates TLS itself with a certificate from our own CA; OpenBao inside the cluster and a browser via port-forward see the same issuer. No DNS, no ingress needed.
3. **Token policies come from the role, identity policies from the group.** Together they are the effective rights. The role gives everyone default; everything beyond that is decided by Keycloak.

Part I – Keycloak in the cluster

Operator

```
V=26.7.4
for f in keycloaks keycloakrealmimports keycloakoidclients keycloaksamlclients; do
    kubectl apply -f https://raw.githubusercontent.com/keycloak/keycloak-k8s-resources/$V/kubernetes/$f.k8s.keycloak.org-v1.yml
done
kubectl apply -n keycloak -f https://raw.githubusercontent.com/keycloak/keycloak-k8s-resources/$V/kubernetes/kubernetes.yml
```

Error 1 – two CRDs too few. The documentation names two CRD files (keycloaks, keycloakrealmimports). With only those two, operator 26.7 crashes at startup:

```
Registered reconciler: 'keycloakoidclientcontroller' for resource:
  'class org.keycloak.operator.crd.v2alpha1.client.KeycloakOIDCClient'
...
Couldn't start informer for keycloakoidclients.k8s.keycloak.org/v2alpha1
```

26.7 has two new CRDs – KeycloakOIDCClient and KeycloakSAMLClient – and the operator registers controllers for them whether you use them or not. Apply all four CRD files of the release version; kustomization.yml in the same directory lists them.

Error 2 – the startup probe. After that the operator kept restarting:

```
Startup probe failed: Get "http://10.42.5.182:8080/q/health/started": connection refused
Container keycloak-operator failed startup probe, will be restarted
...
keycloak-operator 26.7.4 on JVM (powered by Quarkus 3.33.3.2) started in 32.095s
```

The probe allows $5\text{ s} + 3 \times 10\text{ s}$; the JVM needed 32 seconds on this hardware. Just short, every time. Raise failureThreshold to 30 – one patch line on the Deployment, no rebuild.

Database

CloudNativePG is in the cluster; one Cluster object is enough:

```

apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: keycloak-db
  namespace: keycloak
spec:
  instances: 1
  storage:
    size: 5Gi
    storageClass: longhorn
  bootstrap:
    initdb:
      database: keycloak
      owner: keycloak

```

CNPG creates the Secret `keycloak-db-app` with username/password – exactly the shape the Keycloak CR expects. No password in a manifest.

TLS from our own CA

```

apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: keycloak-tls
  namespace: keycloak
spec:
  secretName: keycloak-tls
  privateKey:
    algorithm: ECDSA
    size: 256
    rotationPolicy: Always
  dnsNames:
    - keycloak-service.keycloak.svc.cluster.local
    - keycloak-service.keycloak.svc
  issuerRef:
    kind: ClusterIssuer
    name: openbao-cluster-internal

```

Error 3 – our own role.

```
* subject alternate name keycloak-service.keycloak.svc not allowed by this role
```

The PKI role from N° 5 allows `svc.cluster.local` with subdomains and `demo.svc` – but not `keycloak.svc`. Short names are per namespace; every new namespace that wants short names in its certificate has to be added to `allowed_domains`. That is not a bug, that is the role doing what it is there for. Addition:

```

/ $ bao write pki_int/roles/cluster-internal \
    allowed_domains="svc.cluster.local,demo.svc,keycloak.svc,example.internal" ...

```

`cert-manager` waits with backoff after an error. Whoever does not want to wait deletes the Certificate and recreates it.

The instance

```
apiVersion: k8s.keycloak.org/v2beta1
kind: Keycloak
metadata:
  name: keycloak
  namespace: keycloak
spec:
  instances: 1
  db:
    vendor: postgres
    host: keycloak-db-rw
    database: keycloak
    usernameSecret: { name: keycloak-db-app, key: username }
    passwordSecret: { name: keycloak-db-app, key: password }
  hostname:
    hostname: https://keycloak-service.keycloak.svc.cluster.local:8443
    strict: true
  http:
    tlsSecret: keycloak-tls
  ingress:
    enabled: false
```

The hostname is its own Service name with port. That is the decision that makes the rest simple: OpenBao inside the cluster reaches `keycloak-service.keycloak.svc.cluster.local:8443` directly, and a browser outside reaches the same name via `kubectl port-forward` and an entry in `/etc/hosts`. No ingress, no DNS, and the issuer in the token is identical for both – the condition OIDC otherwise fails on.

```
$ kubectl get keycloak -n keycloak -o jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
True
```

Discovery, from a pod in the cluster, with the root of the OpenBao CA:

```
$ curl --cacert /ca/ca.crt https://keycloak-service.keycloak.svc.cluster.local:8443/realms/master/.well-known/openid-configuration
{"issuer":"https://keycloak-service.keycloak.svc.cluster.local:8443/realms/master", ...}
```

TLS verifies against the CA from N° 5. This is the moment the own PKI pays off.

Part II – The realm as code

```
apiVersion: k8s.keycloak.org/v2beta1
kind: KeycloakRealmImport
metadata:
  name: homelab
  namespace: keycloak
spec:
  keycloakCRName: keycloak
  realm:
    realm: homelab
    enabled: true
    groups:
      - name: openbao-admins
      - name: openbao-readers
    clients:
      - clientId: openbao
        publicClient: false
        secret: openbao-client-secret-123
        standardFlowEnabled: true
        directAccessGrantsEnabled: true      # only for the test without a browser
        redirectUris:
          - "https://bao.example.internal/ui/vault/auth/oidc/oidc/callback"
          - "http://localhost:8250/oidc/callback"
        protocolMappers:
          - name: groups
            protocol: openid-connect
            protocolMapper: oidc-group-membership-mapper
            config:
              claim.name: groups
              full.path: "false"
              id.token.claim: "true"
              access.token.claim: "true"
    users:
      - username: alice
        enabled: true
        email: alice@example.internal
        firstName: Alice
        lastName: Anders
        emailVerified: true
        requiredActions: []
        groups: ["openbao-admins"]
        credentials:
          - type: password
            value: alice123
            temporary: false
      - username: bob
        ...
        groups: ["openbao-readers"]
```

Three decisions in it:

- **The mapper is the core.** Without `oidc-group-membership-mapper` no group is in the token, and OpenBao has nothing to map policies onto. `full.path: "false"` yields `openbao-admins` instead of `/openbao-admins` – the alias in OpenBao must be named exactly like that.

- **Two redirect URIs.** The first for the OpenBao UI (the path is fixed: /ui/vault/auth/oidc/oidc/callback – yes, vault, the UI kept the path), the second for bao login –method=oidc, which opens a local listener on port 8250.
- **emailVerified: true and requiredActions: [].** That is error 4:

```
{"error":"invalid_grant","error_description":"Account is not fully set up"}
```

An imported user without these fields has open required actions (verify e-mail, complete profile) and cannot log in until they are done in a browser. For a test user via password grant there is no such browser.

And a detail you have to know: a KeycloakRealmImport **does not overwrite**. If the realm exists, the import does nothing. After a correction to the manifest: delete the realm via the admin API, delete the CR, recreate the CR.

```
$ kubectl get keycloakrealmimport -n keycloak homelab -o jsonpath='{.status.conditions[?(@.type=="Done")].status}'
True
```

The proof in the token

An ID token for alice via password grant, from a pod in the cluster:

```
$ curl --cacert /ca/ca.crt \
  -d client_id=openbao -d client_secret=openbao-client-secret-123 \
  -d grant_type=password -d username=alice -d password=alice123 -d scope=openid \
  https://keycloak-service.keycloak.svc.cluster.local:8443/realms/homelab/protocol/openid-connect/token
```

The payload of the id_token, decoded:

```
iss:                https://keycloak-service.keycloak.svc.cluster.local:8443/realms/homelab
aud:                openbao
preferred_username: alice
groups:             ["openbao-admins"]
```

Four values that OpenBao needs in a moment: iss must match the oidc_discovery_url, aud must match bound_audiences, preferred_username becomes user_claim, groups becomes groups_claim.

Part III – OIDC in OpenBao

Auth method and configuration

```
/ $ bao auth enable oidc
/ $ bao write auth/oidc/config \
  oidc_discovery_url="https://keycloak-service.keycloak.svc.cluster.local:8443/realms/
homelab" \
  oidc_discovery_ca_pem=@/tmp/root.pem \
  oidc_client_id=openbao \
  oidc_client_secret=openbao-client-secret-123 \
  default_role=default
```

oidc_discovery_ca_pem is where N° 5 and N° 8 touch: OpenBao trusts Keycloak's certificate because it knows its own root CA. Without that line: x509: certificate signed by unknown authority.

Two roles

```
/ $ bao write auth/oidc/role/default \
  role_type=oidc user_claim=preferred_username groups_claim=groups \
  bound_audiences=openbao oidc_scopes=openid \
  allowed_redirect_uris="https://bao.example.internal/ui/vault/auth/oidc/oidc/callback
k,http://localhost:8250/oidc/callback" \
  token_policies=default token_ttl=1h token_max_ttl=8h

/ $ bao write auth/oidc/role/jwt-test \
  role_type=jwt user_claim=preferred_username groups_claim=groups \
  bound_audiences=openbao token_policies=default token_ttl=15m
```

The auth method oidc knows two role types: oidc for the browser flow (redirect, code, callback) and jwt for a token you already have. The second role is only for the test here – it lets anyone with a valid ID token of this realm log in, without a browser. Delete it after the test.

token_policies=default is deliberate: the role grants **no** rights. Those come from the groups.

Policies

```
/ $ bao policy write kv-reader - <<'EOF'
path "secret/data/demo/*"      { capabilities = ["read"] }
path "secret/metadata/demo/*"  { capabilities = ["read", "list"] }
path "secret/metadata/demo"    { capabilities = ["list"] }
EOF
```

admin is the policy from N° 1, Part VI – created in the cluster for the first time here. It does not thereby replace the root token; that is the hardening described in N° 1 and still pending on this cluster.

Group → external group → policy

```
/ $ ACC=$(bao read -field=accessor sys/auth/oidc)
/ $ ID=$(bao write -field=id identity/group name=openbao-admins type=external policies=admin)
/ $ bao write identity/group-alias name=openbao-admins mount_accessor=$ACC canonical_id=$ID
```

The same for openbao-readers → kv-reader. The result:

```
/ $ bao read identity/group/name/openbao-admins
openbao-admins -> ['admin'] | alias: openbao-admins
openbao-readers -> ['kv-reader'] | alias: openbao-readers
```

An **external group** has no members in OpenBao. Its members come from outside: at login OpenBao reads `groups_claim` from the token, looks for a group alias with that name on the auth method's accessor for each value, and attaches the user's entity to that group for the lifetime of the token. The group's policies become `identity_policies`.

The alias name must match the claim value **exactly** – `openbao-admins`, not `/openbao-admins` (hence `full.path: "false"` in the mapper).

A stumbling block from the terminal, not an OpenBao topic: in `zsh`, `GID` is a reserved variable (the process's group ID). `GID=$(bao write -field=id ...)` fails with `bad math expression`. Use another name.

Part IV – The proof

Login with the ID tokens

```
== alice
/ $ bao write -format=json auth/oidc/login role=jwt-test jwt=<id_token>
  token_policies      ['default']
  identity_policies   ['admin']
  effective            ['admin', 'default']

== bob
  token_policies      ['default']
  identity_policies   ['kv-reader']
  effective            ['default', 'kv-reader']
```

Exactly the separation the picture above shows: token_policies from the role (default for both), identity_policies from the group. No password, no role, no user in OpenBao was created for alice or bob.

What the rights do

```
== bob reads a secret:
/ $ BAO_TOKEN=<bob> bao kv get -field=admin_username secret/demo/miniflux
admin
== bob writes:
/ $ BAO_TOKEN=<bob> bao kv put secret/demo/miniflux x=y
* permission denied
== bob lists policies:
* permission denied
== alice lists policies:
admin cert-manager default eso-demo kv-reader snapshot root
```

What OpenBao created

```
/ $ bao list identity/entity/name
entity_0b8b5371.root | aliases ['alice@oidc'] | groups 1
entity_3584c432.root | aliases ['bob@oidc']   | groups 1
entity_...          | aliases ['...@kubernetes'] | groups 0      (ESO, cert-manager, snapsho
t job)
```

On the first login an **entity** with an alias <user_claim>@oidc is created per user. It persists; if the group changes in Keycloak, the entity's group membership changes on the next login – and with it the policies. The three entities without groups are the workloads from N° 2, 5 and 7.

The browser route

For UI and CLI the browser has to reach Keycloak:

```
# /etc/hosts on the workstation
127.0.0.1 keycloak-service.keycloak.svc.cluster.local

$ kubectl port-forward -n keycloak svc/keycloak-service 8443:8443
```

and trust the root CA from N° 5 (browser or operating system). Then:

```
$ export BAO_ADDR=https://bao.example.internal
$ bao login -method=oidc role=default
Complete the login via your OIDC provider. Launching browser to:
  https://keycloak-service.keycloak.svc.cluster.local:8443/realms/homelab/protocol/openi
d-connect/auth?...
Waiting for OIDC authentication to complete...
```

The browser lands at Keycloak, alice logs in, Keycloak redirects to `http://localhost:8250/oidc/callback`, the CLI has the token. In the UI: method **OIDC**, role default, button “Sign in with OIDC Provider”.

This part was not executed here – it needs a browser, sudo for `/etc/hosts` and the trust anchor in the operating system. The role default and the redirect URIs are configured exactly for it; the JWT test in Part IV checks everything except the redirect itself.

Part V – What went wrong, and why

Error	Message	Cause	Fix
1	operator crash: Couldn't start informer for keycloakoidcclients...	26.7 has four CRDs, the docs name two	apply all CRD files of the release
2	failed startup probe, will be restarted	JVM needs 32 s, probe allows 35 s incl. connect-refused phase	raise <code>startupProbe.failureThreshold</code>
3	subject alternate name keycloak-service.keycloak.svc not allowed by this role	PKI role (N° 5) knows <code>demo.svc</code> , not <code>keycloak.svc</code>	add the namespace to <code>allowed_domains</code>
4	Account is not fully set up	imported users with open required actions	<code>emailVerified: true</code> , <code>requiredActions: []</code>
–	bad math expression on <code>GID=...</code>	<code>GID</code> is reserved in <code>zsh</code>	another variable name

And the pattern: three of the four errors came **before** the first OIDC login – operator, certificate, users. OIDC itself worked on the first try, because the four pairs (issuer/discovery, aud/bound_audiences, redirect URIs, claim name) had been checked in the token beforehand. Whoever decodes the ID token before configuring OpenBao is spared the guessing at `error validating token`.

Part VI – Operation

Retiring the temporary admin

Keycloak 26 creates a **temporary** admin on first start (keycloak-initial-admin, user temp-admin). It exists for exactly one task: creating a permanent admin. Then delete the temporary one. Whoever forgets has an admin with a password in a Kubernetes Secret – the very problem this guide solves for OpenBao.

Realm changes

KeycloakRealmImport is an import, not reconciliation. For ongoing changes – new group, new user – there are three routes: the admin console (not as code), the admin REST API in a script, or the new CRs KeycloakOIDCClient (26.7) for clients. Groups and users have no CR yet; for a homelab the console is honest enough, for a company it is Terraform with the Keycloak provider.

Removing the test role

```
/ $ bao delete auth/oidc/role/jwt-test
```

and directAccessGrantsEnabled: false in the client. Together they were the way to test without a browser; in operation they are a bypass of the browser flow – and thus of MFA, if Keycloak requires it.

Nº 1, Part VI, re-read

With OIDC, userpass is no longer the replacement for the root token but only **break glass**: the way in when Keycloak is unreachable. The admin policy now exists; what is missing is the same step as before – prove the replacement access, then revoke root. Only that the replacement access is now called `bao login -method=oidc`, and the break-glass route is userpass with a password in the password manager.

What is still open

- **No MFA.** Keycloak can require OTP; for openbao-admins it should be on. That is Keycloak configuration, not OpenBao.
- **No audit device** (Nº 1). Who logged in via OIDC when is in Keycloak's events – and not in OpenBao.
- **The browser route** is configured, not executed.
- **Client secret in the manifest.** For production: Keycloak generates it, OpenBao receives it via ExternalSecret... from OpenBao. That is a circle that only a manual bootstrap step resolves.

Appendix A – All commands

```
# — Keycloak —————
V=26.7.4
for f in keycloaks keycloakrealmimports keycloakoidclients keycloaksamlclients; do
    kubectl apply -f https://raw.githubusercontent.com/keycloak/keycloak-k8s-resources/$V/kubernetes/$f.k8s.keycloak.org-v1.yml
done
kubectl create ns keycloak
kubectl apply -n keycloak -f https://raw.githubusercontent.com/keycloak/keycloak-k8s-resources/$V/kubernetes/kubernetes.yml
kubectl patch deploy -n keycloak keycloak-operator --type=json \
    -p='[{"op":"replace","path":"/spec/template/spec/containers/0/startupProbe/failureThreshold","value":30}]'
kubectl apply -f k8s/keycloak-db.yaml
bao write pki_int/roles/cluster-internal allowed_domains="svc.cluster.local,demo.svc,keycloak.svc,example.internal" ...
kubectl apply -f k8s/keycloak.yaml
kubectl apply -f k8s/realm-homelab.yaml
kubectl get keycloak,keycloakrealmimport -n keycloak

# — OpenBao (see openbao/oidc-setup.sh) —————
bao read -field=certificate pki/cert/ca > /tmp/root.pem
bao auth enable oidc
bao write auth/oidc/config oidc_discovery_url=... oidc_discovery_ca_pem=@/tmp/root.pem oidc_client_id=openbao oidc_client_secret=... default_role=default
bao write auth/oidc/role/default role_type=oidc user_claim=preferred_username groups_claim=groups bound_audiences=openbao allowed_redirect_uris=... token_policies=default
bao policy write kv-reader - < kv-reader.hcl
ACC=$(bao read -field=accessor sys/auth/oidc)
ID=$(bao write -field=id identity/group name=openbao-admins type=external policies=admin)
bao write identity/group-alias name=openbao-admins mount_accessor=$ACC canonical_id=$ID

# — Test without a browser —————
curl --cacert root.pem -d client_id=openbao -d client_secret=... -d grant_type=password \
    -d username=alice -d password=alice123 -d scope=openid https://keycloak-service.keycloak.svc.cluster.local:8443/realms/homelab/protocol/openid-connect/token
bao write auth/oidc/login role=jwt-test jwt=<id_token>

# — Browser (workstation) —————
kubectl port-forward -n keycloak svc/keycloak-service 8443:8443
bao login -method=oidc role=default
```

Appendix B – Glossary

OIDC – OpenID Connect: login protocol on top of OAuth 2.0 that delivers a signed ID token with claims about the user.

Issuer / discovery – The issuer is the URL that appears in the token as `iss`; `<issuer>/well-known/openid-configuration` holds the configuration including keys. Both must match.

Claim – A field in the token (`preferred_username`, `groups`, `aud`). Keycloak mappers write claims; OpenBao reads them via `user_claim` and `groups_claim`.

Client – The registered application in Keycloak (here `openbao`), with secret and allowed redirect URIs.

Password grant – OAuth flow that exchanges username and password directly for a token. For tests; bypasses MFA.

Role (OIDC auth) – OpenBao object defining how a token is verified and which `token_policies` it gets. `role_type` `oidc` (browser) or `jwt` (existing token).

Entity / alias – OpenBao's identity of a user and its link to an auth method (`alice@oidc`).

External group – Identity group without members of its own; membership comes at login from the `groups_claim`, via a group alias on the auth method's accessor.

identity_policies – Policies a token receives through its entity's groups – in addition to the role's `token_policies`.

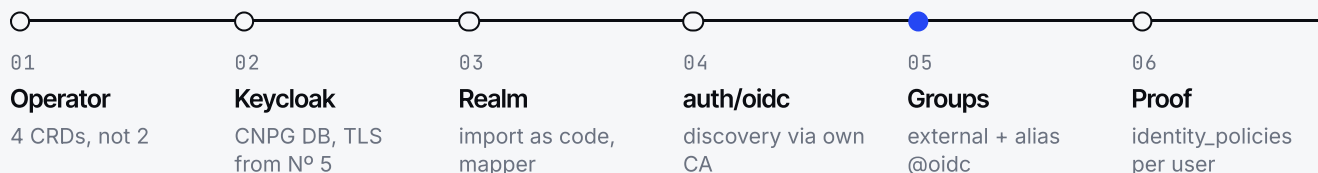
KeycloakRealmImport – CR of the Keycloak operator that imports a realm once. No overwrite, no reconciliation.

Whoever is removed from the group is no admin on the next login. Nobody touches OpenBao.

This guide deploys Keycloak via operator with a CNPG database and a certificate from our own CA, imports a realm as code – client, group mapper, groups, users – and connects OpenBao to it via OIDC. Group membership becomes policies through external groups.

Proven with ID tokens: alice gets `identity_policies [admin]`, bob `[kv-reader]`, bob reads and may not write. Four errors on the way, three of them before the first login: two missing CRDs, a startup probe too tight, our own PKI role, Account is not fully set up.

```
-- Four objects carry the group
Keycloak: group openbao-admins → mapper "groups"
OpenBao:  identity/group openbao-admins (external, policies=admin)
          identity/group-alias openbao-admins @ accessor of auth/oidc
```



keycloak	26.7.4 · operator · realm import
database	CNPG 0.29 · 1 instance
tls	cert-manager · OpenBao CA
openbao	2.6.2 · auth/oidc · identity
policies	admin · kv-reader

20 pages · 6 parts · 2 appendices
Deep dive: **Vault in Practice**
leanpub.com/vault-in-practice
Set in Inter Tight, Inter and JetBrains Mono.
All output is real terminal output.
CC BY 4.0 · share freely · no warranty